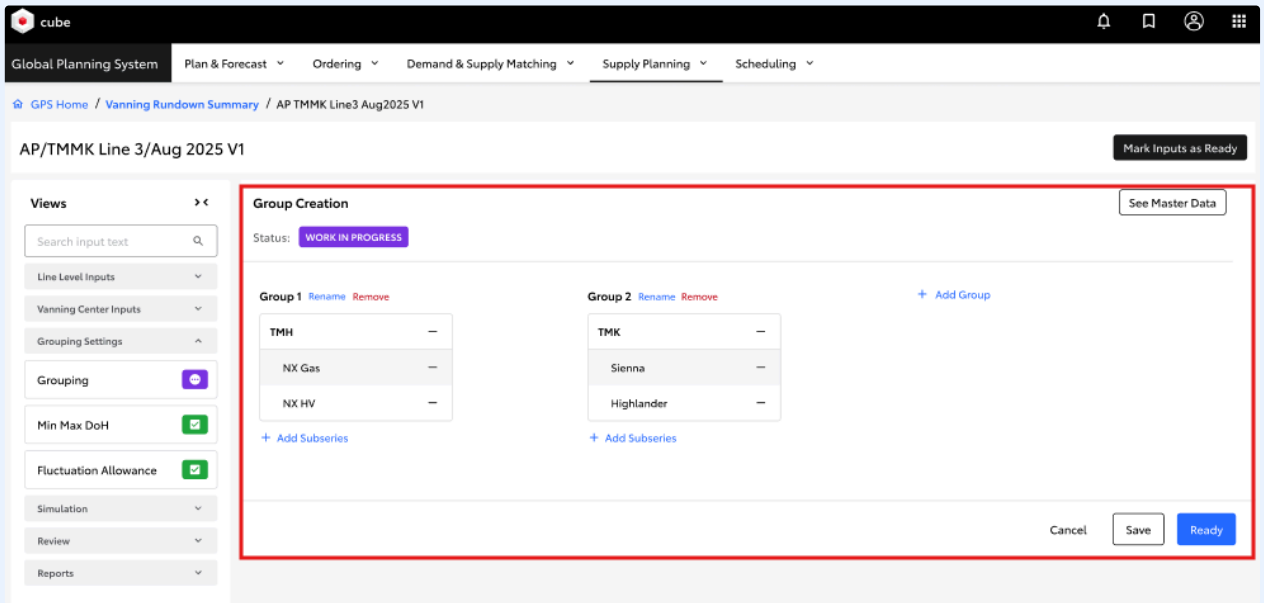
SPPL-197: Grouping Post API technical workflow

Jira Ticket: SPPL-197: BE | Create technical workflow for Grouping Get & Post APIs on parameter inputs page including DB details ACCEPTED

This page provides comprehensive information about the POST API for Grouping on the Input params page, to create/update Groups for the scenario.



Supply Planning POST Groups API Details:

1. **Hosted URL**: https://api.spln.cube<env>.toyota.com/<env>
2. **API Name:** <HOSTED_URL>/v/1/1/sp/groups
3. **Path Param**: N/A
4. **Query Param**: N/A
5. **Method:** POST
6. **Authorization**: Bearer token (**required**)
7. **Lambda Name**: spln-<env>-update-groups-v1-1
8. **API Request Body**:

```
{
  "scenarioId": "uniqueScenarioId",
  "userEmail": "user@toyota.com",
  "data": [
    {
      "groupScenarioMapId": "grp_scenario_mp_id uuid",
      "groupName": "Group1",
      "vanningCenter": "TMK",
      "subSeriesList": ["CAMRY", "RAV4 Gas"],
    },
    {
```

```
      "groupScenarioMapId": 1,//new groups will have integer groupScenarioMapId value
      "groupName": "Group2",
      "vanningCenter": "TMH",
      "subSeriesList": ["HighLander"],
    },
  ]
}
```

9. **API Response:**
   - **Success response with status 200:**

```
{
  "message": "Successfully updated data."
}
```

   - **Error Response:**
     - **any internal server error occurred with status 500:**

```
{
  "errorMessage": "Internal Server Error"
}
```

     - **any unauthorized error occurred with status 401 (Managed by the AWS API Gateway integrated Custom Authorizer):**

```
{
  "message": "Unauthorized"
}
```

     - **If request is in-valid with status 400:**

```
{
  "errorMessage": [<"ValidationError: validation error message”>]
}
```

     - **List of possible validation error message:**
       1. "ValidationError: Request body cannot be empty.”
       2. "ValidationError: scenarioId is required and must be a uuid."
       3. "ValidationError: userEmail is required and must be a string."
       4. "ValidationError: Invalid userEmail."
       5. "ValidationError: Scenario doesn't exist."
       6. "ValidationError: data is required and must be an array with atleast 1 and atmost 15 items."
       7. "ValidationError: groupScenarioMapId is required and must be either a uuid or a non-negative number.”
       8. "ValidationError: groupName is required and must be a unique string."
       9. "ValidationError: vanningCenter is required and must be a string."

10. "ValidationError: subSeriesList is required and must be an array with atleast 1 item."
11. "ValidationError: subSeries must be unique." //check within subSeriesList

10. **Table:**
    a. **scenarios**
    b. **grouping**
    c. **group_namc_line_series_mapper**
    d. **group_scenario_mapper**
    e. **namc_line_subseries**
    f. **scenario_step_status**

SP Update Grouping Data API Workflow:

- **Technical workflow:**
  - Description:
    i. The Supply Planning user logs in to the application.
    ii. After the user provides creates a scenario, opens it, navigates to Grouping and creates/updates group, a request is sent to the SP Post Grouping endpoint: ***sp/groups*** with payload:

```
{
  "scenarioId": "uniqueScenarioId",
  "userEmail": "user@toyota.com",
  "data": [
    {
      "groupScenarioMapId": "grp_scenario_mp_id uuid",
      "groupName": "Group1",
      "vanningCenter": "TMK",
      "subSeriesList": ["CAMRY", "RAV4 Gas"],
    },
    {
      "groupScenarioMapId": 1,//new groups will have integer groupScenarioMapId value
      "groupName": "Group2",
      "vanningCenter": "TMH",
      "subSeriesList": ["HighLander"],
    },
  ]
}
```

    iii. After successful authentication, the **spln-update-groups-v1-1** lambda is triggered by the SP update scenario Grouping endpoint: ***sp/groups***. If authentication fails, an error with status 401 (*refer: error response-**any unauthorized error occurred with status 401***) will be returned and sent to client.
    iv. After user authentication has been confirmed, the **spln-update-groups-v1-1** lambda will perform validation on the request payload. If payload is valid will

proceed to next step else throw validation error (*refer: error response**: Section if request is in-valid with status 400 and List of possible validation error message***). Below query is used to check if provided scenario is valid:

```
select * from supply_planning.scenarios where scenario_id = :scenarioId and
is_active = true;
```

v. After the request payload is successfully validated, the **spln-update-groups-v1-1** lambda execute below logic in a transaction:

1. Constraints (group_id shared across scenarios), the only safe way to make subseries “scenario-specific” is:
   - For an **existing group update only**: deactivate `grp_scenario_mp_id.`
   - For an **existing group update** or a **new group**: you may **reuse** an existing group only if you find one with the **exact same** (namc, line, vanningCenter, subseriesList). Otherwise create new along with its mappers.
2. Deactivate old scenario links (bulk, updates only) - `deactivateScenarioLinks` for update rows (those with `groupScenarioMapId field value`)

```
-- deactivateScenarioLinks: Deactivate scenario→group link(s) (Existing
rows)
UPDATE supply_planning.group_scenario_mapper sgm
    SET
      is_active = FALSE,
      last_updated = NOW(),
      updated_by = ${userEmail}::text
    FROM (
      VALUES
        ${Prisma.join(groupScenarioMapIds.map(id =>
Prisma.sql`(${id}::uuid)`))}
    ) AS v(grp_scenario_mp_id)
    WHERE sgm.grp_scenario_mp_id = v.grp_scenario_mp_id
    AND sgm.scenario_id = ${scenarioId}::uuid
    AND sgm.group_name in (<${groupNames}>)
    AND sgm.is_active = TRUE;
```

3. For each item in the data:
   a. For create or update item in input : Search for existing group with same config (namc, line, vanningCenter, subseriesList).
      i. if found → Reuse that groupId (no namc_line_series_mapper insert) using `findReusableGroupIdExact` below query:

```
-- findReusableGroupIdExact: Use this for update & new group data input
-- check if group with same
namc,line,vanning_center,sub_series_description(list) exist or not
SELECT
    g.group_id
FROM
    supply_planning.grouping g
INNER JOIN
```

```sql
    supply_planning.group_namc_line_series_mapper gnlsm ON g.group_id = gnlsm.group_id
INNER JOIN
    supply_planning.namc_line_subseries nls ON gnlsm.namc_line_series_id = nls.namc_line_series_id
WHERE
    nls.namc = :namc
    AND nls.line = :line
    AND g.vanning_center = :vanning_center
    AND nls.sub_series_description IN (:sub_series_description_list)
    AND g.is_active = 1
    AND gnlsm.is_active = 1
    AND nls.is_active = 1
GROUP BY
    g.group_id
HAVING
    COUNT(DISTINCT nls.sub_series_description) = :count_of_sub_series_description
ORDER BY
    g.last_updated_timestamp DESC
LIMIT 1;
```

ii. if not found → Create a new group and namc_line_series_mapper record with groupId i.e using below queries `insertGroupingReturningId` (fork) → `insertMappingsForNewGroup`

```sql
-- insertGroupingReturningId: Create a new group records(for create-when-no-reuse available). Returns: group_id
INSERT INTO supply_planning.grouping (
      vanning_center,
      effective_in,
      is_active,
      created_by
    )
    VALUES (
      ${vanningCenter}::text,
      NOW()::date,
      TRUE,
      ${userEmail}::text
    )
    RETURNING group_id;

-- insertMappingsForNewGroup: Create mappings for a group_id (replace semantics for the NEW group only)
--For new group_id, insert/activate mappings for subSeriesList. (No deactivation needed since it’s a newly created group_id.)
WITH input_subs AS (
      SELECT * FROM (
        VALUES
          ${Prisma.join(subSeriesList.map(s => Prisma.sql`(${s}::text)`))}
      ) AS v(sub_series_description)
    ),
    resolved AS (
      SELECT DISTINCT nls.namc_line_series_id
      FROM input_subs i
      JOIN supply_planning.namc_line_subseries nls
        ON nls.sub_series_description = i.sub_series_description
      WHERE nls.is_active = TRUE
    )
    INSERT INTO supply_planning.group_namc_line_series_mapper (
      group_id,
      namc_line_series_id,
      is_active,
```

```
      created_by
    )
    SELECT
      ${groupId}::uuid,
      r.namc_line_series_id,
      TRUE,
      ${userEmail}::text
    FROM resolved r
    ON CONFLICT (group_id, namc_line_series_id)
    DO UPDATE SET
      is_active = TRUE,
      last_updated_timestamp = NOW(),
      updated_by = ${userEmail}::text;
```

4. Collect all chosen/created groupIds → `upsertScenarioLinksBulk`

```
-- upsertScenarioLinksBulk: For both inserting new group scenario links and updating exitsing links
INSERT INTO supply_planning.group_scenario_mapper (
      scenario_id,
      group_id,
      group_name,
      is_active,
      created_by
    )
    VALUES
      ${Prisma.join(groupIds.map(gid => Prisma.sql`(
        ${scenarioId}::uuid,
        ${gid}::uuid,
        ${groupName}::text,
        TRUE,
        ${userEmail}::text
      )`))}
    ON CONFLICT (scenario_id, group_id, group_name)
    DO UPDATE SET
      is_active = TRUE,
      last_updated_timestamp = NOW(),
      updated_by = ${userEmail}::text;
```

vi. Update the step & scenario status to “In Progress”. Only update scenario status to “In Progress” if its not already using scenarioData from query (iv) response.

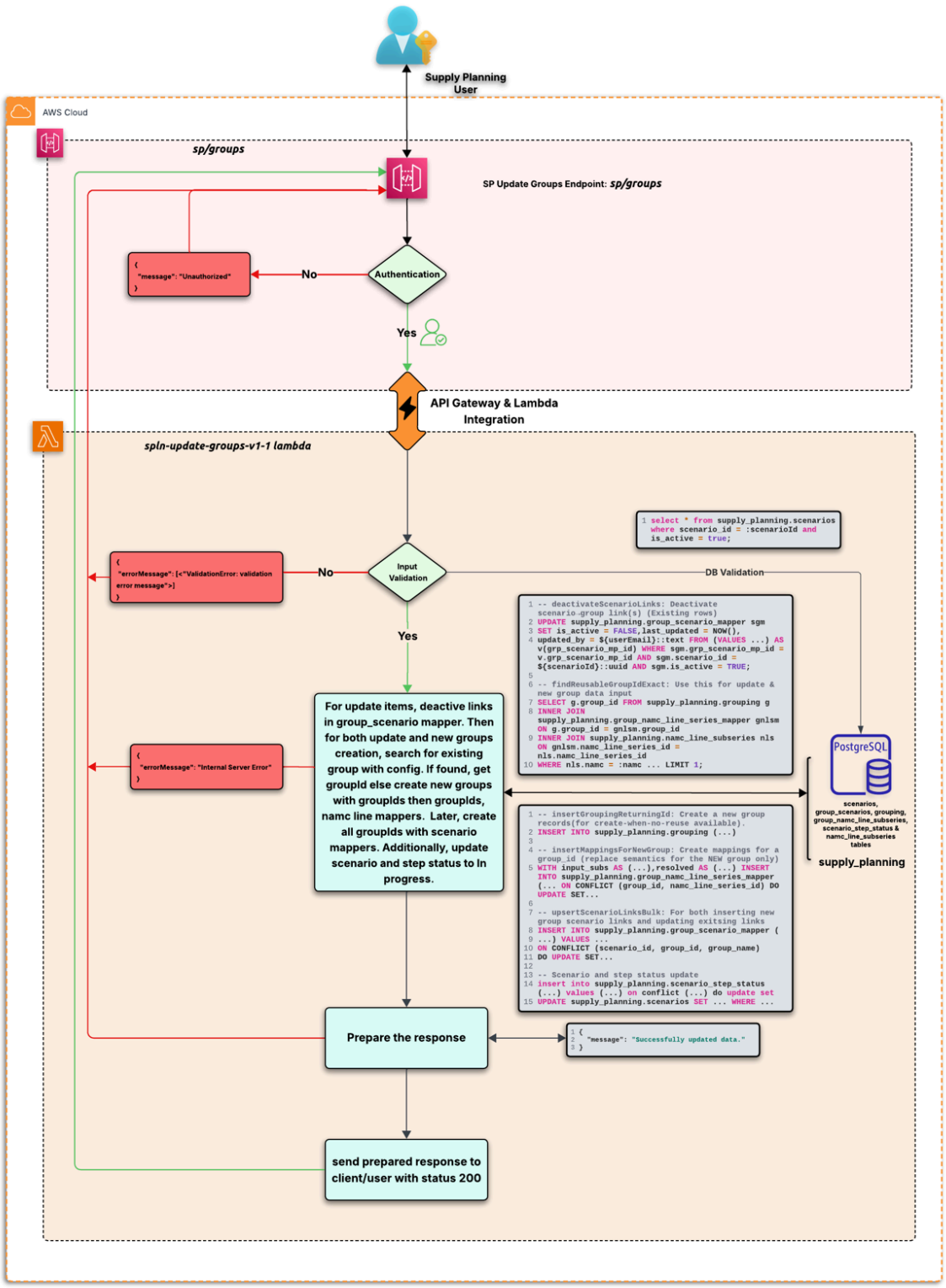
```
insert into supply_planning.scenario_step_status (scenario_id, input_step_type, input_step_name, status, updated_by, last_updated_timestamp)
values (${scenarioId}::uuid, ${stepType}, ${stepName}, ${status}, ${userEmail}, now())
on conflict (scenario_id, input_step_name)
do update set status = ${status}, updated_by = ${userEmail}, last_updated_timestamp = now()
returning scenario_step_status_id::uuid;

UPDATE supply_planning.scenarios
SET scenario_status = "In Progress",
    last_updated_timestamp = CURRENT_TIMESTAMP
WHERE scenario_id = ${scenarioId}::uuid
returning scenario_id;
```

vii. After upserting the Grouping & status data, return the below response to the client with a status code of 200:

```
{
  "message": "Successfully updated data."
```

```
}
```

viii. If there is an error while fetching the data, an “Internal Server Error” with a status code of 500(*refer: error response-**any internal server error occurred with status 500***) will be returned. Otherwise, a successful response with a status code of 200 will be sent.



SP Groups Post API Techncial workflow

Here's an example showing exactly what happens in DB for your POST workflow (reuse on create, fork on update):

Lets assume we have below payload with 2 updates & 1 create:

```
{
  "scenarioId": "S1",
  "userEmail": "user@toyota.com",
  "data": [
    {
      "groupScenarioMapId": "SGM1",
      "groupName": "Group1",
      "vanningCenter": "TMK",
      "subSeriesList": ["CAMRY"]
    },
    {
      "groupScenarioMapId": "SGM2",
      "groupName": "Group2",
      "vanningCenter": "TMX",
      "subSeriesList": ["CAMRY", "RAV4 Gas"]
    },
    {
      "groupScenarioMapId": 1,
      "groupName": "Group3New",
      "vanningCenter": "TMH",
      "subSeriesList": ["HighLander"]
    }
  ]
}
```

Starting DB State (before POST):

- **group_scenario_mapper table** (for scenario S1) - (Note: Other scenarios may also link to G1 / G2 (shared groups)).
- *TODO: Maintain groupName in group_scenario_mapper inorder to reuse a group for another scenario grouping data*

grp_scenario_mp_id	scenario_id	group_id	is_active
SGM1	S1	G1	true
SGM2	S1	G2	true

- **grouping table** - (G3 Group3New with TMH and HighLander subSeries may or may not exist)
- *TODO: Remove groupName in grouping table inorder to reuse a group for another scenario grouping data*

group_id	group_name	vanning_center
G1	Group1	TMK
G2	Group2	TMX
G3	Group3New	TMH

- **group_namc_line_series_mapper table** - (global mappings for groups & subSeries)

Assume:

- CAMRY = N1
- RAV4 Gas = N2
- HighLander = N3

group_namc_line_series_id	group_id	namc_line_series_id	subSeries	is_active
M1	**G1**	**N1**	CAMRY	true
M2	**G1**	**N2**	RAV4 Gas	true
M3	**G2**	**N1**	CAMRY	true
M4	**G3**	**N3**	HighLander	true

Example current active mappings:

- `G1` is globally "Group1/TMK" and currently has **CAMRY + RAV4 Gas**
- `G2` is globally "Group2/TMX" and currently has **CAMRY**
- `G3` is globally "Group3New/TMH" and already has **HighLander** (this is the "reuse exists" case)

Step-by-step what happens:

Step 1: Bulk deactivate old scenario links (updates only)

We pass `groupScenarioMapIds = [SGM1, SGM2]`.

After this step, in `group_scenario_mapper` table for S1: (No impact on other scenarios using G1 or G2.)

grp_scenario_mp_id	scenario_id	group_id	is_active
SGM1	S1	G1	**false**
SGM2	S1	G2	**false**

Step 2: Process each item in payload data field

Item 1 (update SGM1): reuse or fork (instead of creating a new group directly for updates, find existing group by NAMC, Line, Vanning Center & Subseries. If not found, create new group with mappers. If found, get groupId.)

You **always check for a group with same config, if found use it else fork(create new record)** for updates.

1. `findReusableGroupIdExact` search for group with same NAMC, Line, Vanning Center and Subseries
   a. if found, get the groupId (Lets say GX)
   b. else `insertGroupingReturningId` creates a new group row → **grouping table i.e.** New group_id: **G1F** (fork of Group1/TMK)
      i. `insertMappingsForNewGroup` creates mappings for G1F with subSeriesList = [CAMRY] → **group_namc_line_series_mapper table i.e.** Insert (G1F, N1) active

Now:

- Scenario S1 will later point to GX or G1F → **group_scenario_mapper table**
- Other scenarios still pointing to G1 see their original mappings unchanged

Item 2 (update SGM2): reuse or fork (instead of creating a new group directly for updates, find existing group by NAMC, Line, Vanning Center & Subseries. If not found, create new group with mappers. If found, get groupId.)

Again same as Item 1.

1. `findReusableGroupIdExact` search for group with same NAMC, Line, Vanning Center and Subseries
   a. if found, get the groupId (Lets say GY)
   b. else `insertGroupingReturningId` creates a new group row → **grouping table i.e.** New group_id: **G2F** (fork of Group2/TMX)
      i. Mappings for G2F with subSeriesList = [CAMRY, RAV4 Gas] → **group_namc_line_series_mapper table i.e.** Insert (G2F, N1) active & (G2F, N2) active

Now:

- Scenario S1 will later point to GY or G2F → **group_scenario_mapper table**
- Other scenarios still pointing to G2 see their original mappings unchanged

Item 3 (create): If an exact match already exists in grouping + namc_line_series mapper table, we reuse it; otherwise we create it.

Run `findReusableGroupIdExact` for:

- *(TMMI, Line1, TMH, [HighLander])*

Two cases:

Case A: exact match exists

Suppose group **G3** already exists with:

- namc = TMMI
- line = Line1
- vanning_center = TMH
- active mappings exactly = {HighLander}

Then you reuse:

- chosen group_id = **G3**
- no new grouping row → **grouping table**
- no new mapper rows → **group_namc_line_series_mapper table**

Now:

- Scenario S1 will later point to G3 with input group name → **group_scenario_mapper table**

Case B: no match exists

Then you create:

- new group_id: **G3N** → **grouping table**
- insert mapping (G3N, N3) active: → **group_namc_line_series_mapper table**

Now:

- Scenario S1 will later point to G3N with input group name → **group_scenario_mapper table**

Summary of above steps for **group_namc_line_series_mapper table:**

1. Update Group1/TMK to only [CAMRY] for scenario S1 → reuse or fork new group i.e. **GX** or **G1F** with only CAMRY
2. Update Group2/TMX to [CAMRY, RAV4 Gas] for scenario S1 → reuse or fork new group i.e. **GY** or **G2F** with CAMRY + RAV4
3. Create Group3New/TMH [HighLander]
   a. ***reuse existing G3 (so no new mappings)***

group_namc_line_series_id	group_id	namc_line_series_id	subSeries	is_active
M1	**G1**	N1	CAMRY	true
M2	**G1**	N2	RAV4 Gas	true
M3	**G2**	N1	CAMRY	true
M4	**G3**	N3	HighLander	true
M5	**G1F**	**N1**	CAMRY	true
M6	**G2F**	**N1**	CAMRY	true
M7	**G2F**	**N2**	RAV4 Gas	true

b. If Group3New/TMH did NOT exist (no reuse case) - Then you would create **G3N** and add one mapping row:

group_namc_line_series_id	group_id	namc_line_series_id	subSeries	is_active
M8	**G3N**	**N3**	HighLander	true

Step 3: Bulk upsert scenario links for chosen groupIds

You now link S1 to the "new chosen" group_ids:

- For item 1: **GX** (reuse) or **G1F** (new)
- For item 2: **GY** (reuse) or **G2F** (new)
- For item 3: **G3** (reuse) or **G3N** (new)

After upsert, `group_scenario_mapper` for S1 looks like:

grp_scenario_mp_id	scenario_id	group_id	is_active
SGM1	S1	G1	false
SGM2	S1	G2	false
SGM_NEW1	S1	**G1F**	true
SGM_NEW2	S1	**G2F**	true
SGM_NEW3	S1	**G3 or G3N**	true

Now GET Groups API returns below data for scenario S1:

Using your GET SQL query (joins through `group_scenario_mapper`):

- Group1/TMK (**GX or G1F**) → only **CAMRY**
- Group2/TMX (**GY or G2F**) → **CAMRY, RAV4 Gas**
- Group3New/TMH (**G3 or G3N**) → **HighLander**

And importantly:

- Any other scenario that used **G1** or **G2** still sees their old subseries, unchanged.

Note:

We will be utilizing this API for both to update group data and create new groups.

This API doesn't include delete group feature. It will be a separate API.

DB constraints:

To make `ON CONFLICT` work in mapper:

- `group_scenario_mapper: UNIQUE(scenario_id, group_id, group_name)`
- `group_namc_line_series_mapper: UNIQUE(group_id, namc_line_series_id)`

`Also ensure group_scenario_mapper has is_active BOOLEAN and groupName is part of group_scenario_mapper table instead of grouping table.`

Queries:

What is the maximum number of groups user can create per scenario? 15 @Daniel Diago